Rahul Rangwani
Problem Set 1
January 17, 2020
Intro to Machine Learning

Statistical and Machine Learning

On a very high level, the difference between supervised and unsupervised learning has to do with the type of data and outcomes produced. With supervised learning, we generally have more structured and well-labelled data. Using this data, we are able to train our model of choice, and then make predictions on data where we haven't seen the outcomes yet, forecasting these unseen values. With unsupervised learning, it generally deals with less structured and unlabeled data, where our input data doesn't have any corresponding output information.

In supervised learning, our training data consists of inputs and outputs. For each set of X values, we have a corresponding Y value. The goal of supervised learning, then, is given new X values, to determine the Y value of best fit, whether it be a numerical value or a categorical classification. This relationship can be linear, quadratic, or some take some other form. With unsupervised learning, however, we don't have this type of relationship. Generally, there only exists X values in unsupervised models, and our goal is to describe some sort of relationship between the given data points. This is much less structured than supervised learning, and can be used for more complex and experimental models with less defined shape.

When we use these models, we have different goals for each. With a supervised model, our target output is a set of predicted Y values for a set of known X values, produced from a model trained with sets of known X and Y value pairs. We usually try to get to two different types of outputs here – numerical values or categorical classifications. This is useful when forecasting or predicting future values on a dataset, or trying to classify categorical observations. With unsupervised learning, we can have many goals. One of the most common reasons to use unsupervised learning is to simply find patterns in the data we might not be able to see just by looking. One way to do this is with clustering, which finds commonalities in the data and groups the data by these common features. We hope to find some sort of structure in the very unstructured data that we have.

Both types of algorithms have their advantages and disadvantages. With supervised learning, we can predict future outcomes using past data, but we can also be prone to overfitting our model to the training data. This brings in the question of balance – we can't focus on training the model to fit exactly the training data at the cost of making bad predictions, but we also have to fit the model well enough to work effectively. With unsupervised learning, we have the ability to work with unstructured data, which is much more available and easy to acquire. We can also find patterns in the data we wouldn't necessarily see when doing regular analysis, as unsupervised algorithms can be more exploratory in nature. However, a problem with this is that because there is no specific outcome we are predicting, it may take long to tweak and modify the model we use to find meaningful results. Clustering algorithms can also produce super granular results such that each observation can be it's own cluster if our models are not set properly.

Supervised and unsupervised learning are very different, and each has their own applications. It is helpful to have both types of models available, as depending on our data and goals, we have options on both sides to use.

Linear Regression

a. The code used here was:

```
lmMod <- lm(mtcars$mpg ~ mtcars$cyl, data=mtcars); summary(lmMod)
```

The output found was:

```
Residuals:
    Min      1Q  Median      3Q     Max 
-4.9814 -2.1185  0.2217  1.0717  7.5186 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  37.8846     2.0738   18.27  < 2e-16 ***
mtcars$cyl   -2.8758     0.3224   -8.92 6.11e-10 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 3.206 on 30 degrees of freedom
Multiple R-squared:  0.7262,	Adjusted R-squared:  0.7171 
F-statistic: 79.56 on 1 and 30 DF,  p-value: 6.113e-10
```

Our b_0 parameter is **37.8846** and our b_1 coefficient is be **-2.8758**

b. Our population regression function is:

$$mpg = 37.8846 - 2.8758 * cyl$$

This tells us that with 0 cylinders, a car would get 37.8846 mpg, and with each additional cylinder we lose 2.8758 mpg.

c. I executed this code:

```
lmModwt <- lm(mtcars$mpg ~ mtcars$cyl + mtcars$wt, data=mtcars); summary(lmModwt)
```

And got this output:

```
Residuals:
    Min      1Q  Median      3Q     Max 
-4.2893 -1.5512 -0.4684  1.5743  6.1004 

Coefficients:
            Estimate Std. Error t value Pr(>|t|)    
(Intercept)  39.6863     1.7150  23.141  < 2e-16 ***
mtcars$cyl   -1.5078     0.4147  -3.636 0.001064 ** 
mtcars$wt    -3.1910     0.7569  -4.216 0.000222 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.568 on 29 degrees of freedom
Multiple R-squared:  0.8302,	Adjusted R-squared:  0.8185 
F-statistic: 70.91 on 2 and 29 DF,  p-value: 6.809e-12
```

Our new regression equation is:

$$mpg = 39.6863 - 1.5078 * cyl - 3.1910 * wt$$

What this tells us is that weight reduces the impact of cylinders on mpg from what we originally thought. Our coefficient for cylinder is almost half of what it was originally, and weight has about double the effect on mpg as cylinders.

d. I executed this code:

```
lmModint <- lm(mtcars$mpg ~ mtcars$cyl*mtcars$wt, data=mtcars); summary(lmModint)
```

And got this output:

```
Residuals:
    Min      1Q  Median      3Q     Max 
-4.2288 -1.3495 -0.5042  1.4647  5.2344 

Coefficients:
                     Estimate Std. Error t value Pr(>|t|)    
(Intercept)           54.3068     6.1275   8.863 1.29e-09 ***
mtcars$cyl            -3.8032     1.0050  -3.784 0.000747 ***
mtcars$wt             -8.6556     2.3201  -3.731 0.000861 ***
mtcars$cyl:mtcars$wt   0.8084     0.3273   2.470 0.019882 *  
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 2.368 on 28 degrees of freedom
Multiple R-squared:  0.8606,	Adjusted R-squared:  0.8457 
F-statistic: 57.62 on 3 and 28 DF,  p-value: 4.231e-12
```

In this case, both of the coefficients for cylinder and weight are much higher than without the interaction term. With the relative difference between each coefficient and the interaction term (where the coefficient is .8084), we see that that there is not too much interaction between weight and cylinders that might impact the mpg. For each discrete number of cylinders, the effect of weight on mpg would now be:

$$-8.6556 + .8084 * cyl_x$$

And similarly for each discrete weight of car, the effect of cylinder on weight would be:

$$-3.8032 + .8084 * wt_x$$

Non-Linear Regression

a. I executed this code:

```
polymod <- lm(wagedat$wage ~ poly(wagedat$age, 2), data=wagedat); summary(polymod)
```

And got these results:

```
Residuals:
    Min      1Q  Median      3Q     Max 
-99.126 -24.309  -5.017  15.494 205.621 

Coefficients:
              Estimate Std. Error t value Pr(>|t|)    
(Intercept) -10.425224   8.189780  -1.273    0.203    
age           5.294030   0.388689  13.620   <2e-16 ***
I(age^2)     -0.053005   0.004432 -11.960   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 39.99 on 2997 degrees of freedom
Multiple R-squared:  0.08209,	Adjusted R-squared:  0.08147 
F-statistic:   134 on 2 and 2997 DF,  p-value: < 2.2e-16
```

The regression equation is:

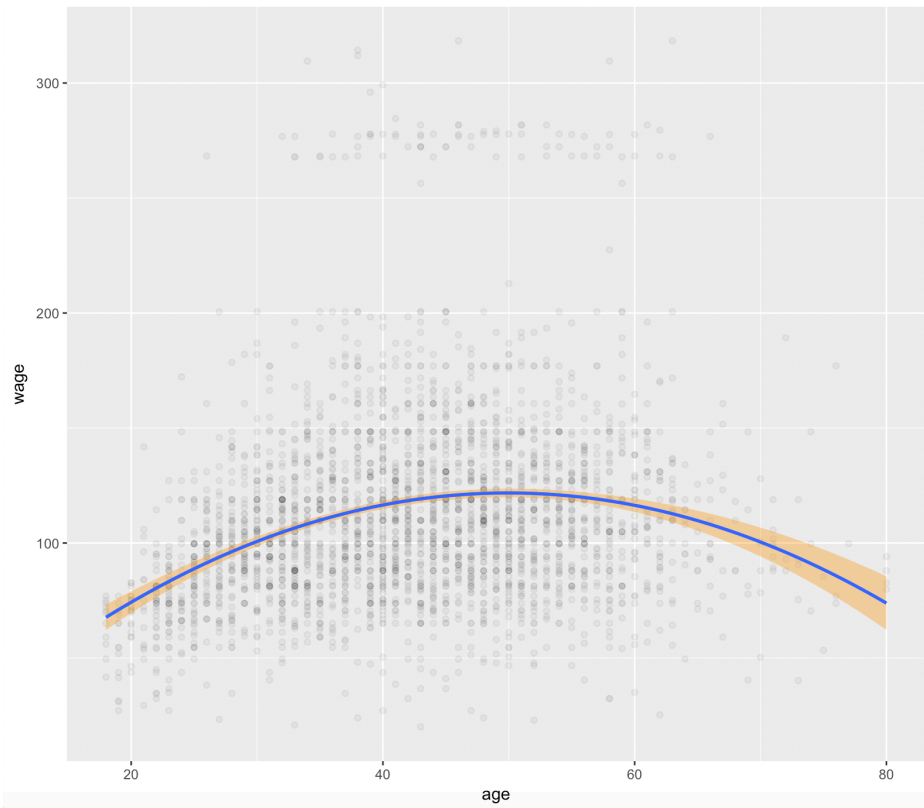
$$wage = -10.425224 + 5.294030 * age - 0.053005 * age^2$$

Because the coefficient on the squared term is negative, the curve would end up looking concave, but the curve would be fairly flat due to the low absolute value of the coefficient. What this tells us is that while increasing age generally increases income, after a certain age, the wage starts to decrease at a slow rate, at around 50 years of age.

b. To generate the plot, I executed this code:

```
ggplot(wagedat, aes(y=wage, x=age)) +
  geom_point(alpha = .05) +
  stat_smooth(method = "lm",fill = 'orange', formula = y ~ poly(x,2))
```

Plot:



c. With this graph, we can see that the error bars are larger on both ends of the curve. This indicates that there is more uncertainty with the wage predictions as age deviates from the average, especially on the older end. The older one gets, after about 50 (the top point of the curve), wage goes down but there is much less confidence in the wage prediction. This may be due to fewer data points and more outliers, as people stop working and some may advance to very senior positions to earn very large amounts. By using a polynomial regression, we are implying that this data would not fit well with a linear regression. The change in slope from positive to negative would not be shown in a linear model, and so we would not be able to see that wage actually would decrease after a certain point.

d. With linear regressions, we are only looking for one type of relationship – linear. While many many linear relationships do exist in data, there are also many other types of non-linear relationships to explore. Using a polynomial model allows us to explore these relationships and see if any other types of curves would work better, like this case with wage and age, where at first glance a linear model might seem like a good fit, but then when you apply a polynomial model the fit is much better. However, with polynomial models, depending on the order of the model, outliers can cause big deviations in the actual curve shape and reduce the accuracy of the model as a whole. A key difference is that a linear model might be more rigid and can only fit linear data, but might be more robust to outliers depending on the dataset. Despite this, a non-linear model has more flexibility in the types of relationships one can determine from using one.